
Training-Free $O(N \log N)$ Sparse Attention via Recursive Block Selection

Thomas Zeiringer

Institute of Automation and Control
Graz University of Technology
Graz, Austria
thomas.zei9@gmail.com

Abstract

Recursive Block-Selection Attention (RBA), a training-free sparse attention mechanism, is proposed. It organizes the key-value cache into a multi-scale hierarchy of summed key composites and selects relevant pairs by a top- k descent from coarse to fine, reducing compute and memory traffic from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$ while filtering the cache rather than evicting or compressing it. As a drop-in replacement in a frozen Gemma-4-31B-it, RBA matches or exceeds dense attention across all 48 Needle-in-a-Haystack configurations – most decisively on mid-context needles, where dense attention exhibits the lost-in-the-middle effect – while adding under 7.5% memory over a standard KV cache. No optimized kernel was implemented, so timing is not reported, and no comparison to other sparse attention methods is made. Code: <https://github.com/Thomas23579/RecursiveBlockSelectionAttention>.

1 Introduction

At context windows of 10^5 – 10^6 tokens, the attention mechanism in Transformers exhausts accelerator memory and dominates inference latency. Both problems stem from the key-value (KV) cache, which grows linearly with the context. During prefill, every input token attends to all preceding tokens, for a quadratic total cost; during decoding, each generated token must read the entire cache, rendering attention memory-bandwidth bound.

A second, subtler problem compounds the first. As the number of attended tokens grows, the softmax normalization spreads probability mass over an ever larger set, a phenomenon called *attention dispersion*. The resulting near-uniform attention weights wash out the distinction between relevant and irrelevant tokens, degrading model capability independently of any computational constraint [1, 2].

This work proposes *Recursive Block-Selection Attention* (RBA), a sparse attention method that organizes the KV cache into a multi-scale hierarchy: groups of keys are recursively summarized into composite representatives, yielding progressively coarser views of the context. To select relevant key-value pairs, the hierarchy is traversed from the coarsest scale down to the original keys, retaining at each scale only the top- k composites by attention score – a truncated breadth-first search. The softmax is computed over the retained frontier alone, a small subset of the full key set.

This construction yields four properties. First, because the hierarchy has logarithmic depth and each scale contributes a frontier of bounded size, every query costs $\mathcal{O}(\log N)$ in both compute and memory traffic; over a full sequence this amounts to $\mathcal{O}(N \log N)$. Dense attention, by contrast, costs $\mathcal{O}(N)$ per query and $\mathcal{O}(N^2)$ overall. Second, restricting the softmax to a frontier of bounded size limits attention dispersion by construction. Third, the KV cache is *filtered* rather than evicted or compressed – every key-value pair remains available at every decoding step, so no information is permanently

lost. Fourth, the method requires no training or fine-tuning: it is a drop-in replacement for the dense attention of a frozen pretrained model.

Related Work Long-context capability itself is addressed by methods orthogonal to this work: YaRN extends rotary position embeddings beyond the pretraining context length [3], and Scalable-Softmax (SSMax) mitigates attention dispersion by scaling the softmax with the context length [2]. These methods make long contexts *usable*; they do not make them *cheap*. Dense attention still incurs quadratic prefill cost and a per-token cache read that grows linearly with the context. The methods below – including the proposed one – target this cost and some also mitigate attention dispersion.

Recent work tackles long-context inference efficiency along three lines: KV compression, KV eviction, and sparse attention. KV compression shrinks the cache’s memory footprint: Dynamic Memory Compression (DMC) learns whether to append a new KV pair or merge it into the preceding one [4]; Grouped-Query Attention (GQA) shares key–value heads across groups of query heads [5], and Cross-Layer Attention (CLA) shares them across layers [6]; quantization reduces the bit-width of stored keys and values [7]; Multi-head Latent Attention (MLA) caches a low-rank latent vector in place of the full keys and values, trading extra compute for reduced memory bandwidth [8].

KV eviction shortens the cache by discarding key–value pairs outright: sliding-window attention attends only to a fixed window of recent tokens, dropping the rest [9]; Heavy-Hitter Oracle (H2O) retains the tokens with the highest accumulated attention scores [10]; SnapKV retains, during prefill, the tokens most attended to by an observation window at the end of the prompt [11].

Sparse attention restricts each query to a selected subset of keys: Quest, InfLLM-V2, and Mixture of Block Attention (MoBA) select key–value pairs at block granularity, ranking contiguous blocks by lightweight block-level scores and attending only to the top-ranked ones [12–14]; Native Sparse Attention (NSA) likewise partitions keys into contiguous blocks but employs a learnable MLP to compress each block into a summary representation that guides selection [15]; Hierarchically Pruned Attention (HiP) instead refines coarse blocks recursively into ever-finer neighborhoods [16]; DeepSeek Sparse Attention (DSA) scores individual tokens with a low-dimensional indexer trained jointly with the model and computes exact attention over the top- k entries [17]; MiniMax Sparse Attention (MSA) adopts the same decoupled-indexer design but selects at block granularity, sharing the selection across each query group [18].

The proposed method belongs to this third line: the hierarchy serves purely as a selection structure, while the attended keys and values remain the original, unmodified ones. It is most closely related to HiP, differing in that each block is represented by a composite of its keys rather than its center key. The remaining methods are referred to as *flat* sparse attention methods: their partitioning has a single level of granularity, unlike the multi-level structure of HiP and the proposed RBA. As a result, the flat methods incur linear per-token complexity, whereas the hierarchical ones achieve sublinear per-token complexity.

2 Methodology

2.1 Dense Attention

Given an input sequence of token representations $x_1, \dots, x_N$, each attention head computes key, query, and value projections

$$k_j = W_K x_j, \quad q_i = W_Q x_i, \quad v_j = W_V x_j, \quad (1)$$

where i indexes the current token and $j \leq i$ indexes the tokens up to and including it, per causal masking; batch and head indices are omitted for clarity. The scaled dot-product score between query i and key j is

$$s_{i,j} = \frac{k_j^\top q_i}{\sqrt{d_h}}, \quad (2)$$

where d_h is the head dimension; the $\sqrt{d_h}$ scaling is conventional, though this factor may also be learned. The scores are mapped onto the probability simplex $\Delta = \{a \geq 0 : \sum_j a_j = 1\}$ by the softmax,

$$a_{i,j} = \text{softmax}_{j \leq i}(s_{i,j}) = \frac{\exp(s_{i,j})}{\sum_{j' \leq i} \exp(s_{i,j'})}, \quad (3)$$

and the head output is a weighted combination of the values,

$$y_i = \sum_{j \leq i} a_{i,j} v_j. \quad (4)$$

At long context lengths, softmax exhibits a problematic property: it assigns strictly positive weight to every token, since $\exp(s) > 0$ for all finite s . As the number of keys grows, more terms enter the normalizing denominator and the distribution flattens, diluting the contribution of the few genuinely relevant tokens – the dispersion effect discussed in the introduction.

2.2 Recursive Block-Selection Attention (RBA)

Intuitive preliminary. The observation motivating the proposed approach is that adding uncorrelated keys to a sum leaves the expected attention score of any relevant component unchanged.

Consider a query q and two keys k_1, k_2 . Decompose $k_1 = \alpha q + e_1$ into a component aligned with the query and a residual e_1 , and let $k_2 = e_2$ be uncorrelated with q , i.e. $\mathbb{E}[e_1^\top q] = \mathbb{E}[e_2^\top q] = 0$. Then

$$\mathbb{E}[(k_1 + k_2)^\top q] = \mathbb{E}[(\alpha q + e_1)^\top q] + \mathbb{E}[e_2^\top q] = \alpha \mathbb{E}[\|q\|^2] = \mathbb{E}[k_1^\top q]. \quad (5)$$

In expectation, adding an irrelevant key to a relevant one does not change the score seen by the query. Conversely, if no key in a group is correlated with the query, the group sum scores low, correctly signaling irrelevance. This suggests using the sum of a group’s keys as its representative.

The sum of B keys equals B times their average, so the composite score is, up to a uniform factor, the average attention score of the group; Lee et al. [16] observed that nearby keys tend to be similar, which makes this average a faithful gauge of each member’s relevance. Since all groups share the same size, the factor B is common to all composites and leaves their ranking unaffected. Where similarity degrades, as it must once groups grow large, (5) provides a complementary guarantee: members uncorrelated with the query contribute nothing to the expected composite score, so the relevant components are not masked by the irrelevant remainder.

Two caveats apply. First, keys whose alignment coefficients have opposite signs interfere destructively: for $k_1 = \alpha q + e_1$ and $k_2 = -\beta q + e_2$ with $\alpha, \beta > 0$, $\mathbb{E}[(k_1 + k_2)^\top q] = (\alpha - \beta) \mathbb{E}[\|q\|^2]$, so a relevant key may be masked within its group. The similarity of nearby keys renders opposite-signed alignments within a block unlikely. Second, the guarantees above hold in expectation only: the residuals of a group’s keys accumulate, so the variance of the composite score grows with group size while the relevant signal does not, and selection becomes less reliable at very coarse scales.

Hierarchy structure. Let B denote the branching factor (block size), C the candidate budget of the selection (introduced below), and index the hierarchy by $\ell = 0, 1, \dots, H - 1$, where $\ell = 0$ is the original token scale. Scale ℓ holds $\lceil N/B^\ell \rceil$ entries, indexed by $0 \leq j < \lceil N/B^\ell \rceil$; for $\ell \geq 1$ these are composite keys, each formed by summing B keys from scale $\ell - 1$:

$$k_j^{[\ell]} = \sum_{j'=jB}^{jB+B-1} k_{j'}^{[\ell-1]}, \quad \ell = 1, \dots, H - 1, \quad (6)$$

where $k_j^{[0]} = k_j$ are the original keys. An incomplete group is padded with zeros, so every entry is represented in the hierarchy and no special treatment during selection is required. The depth $H = \lceil 1 + \log_B(N/C) \rceil$ is chosen so that the coarsest scale holds at most C entries. Parent and child indices are related by

$$j^{[\ell]} = \lfloor j^{[\ell-1]} / B \rfloor \quad \text{and} \quad j^{[\ell-1]} \in \{j^{[\ell]}B + b : 0 \leq b < B\}. \quad (7)$$

Attention. Attention proceeds top-down through the hierarchy. At decode step i , scale ℓ holds $n_\ell = \lceil (i + 1)/B^\ell \rceil$ entries. At each scale, the query scores a candidate set $\mathcal{J}_i^{[\ell]}$,

$$s_{i,j}^{[\ell]} = k_j^{[\ell]\top} q_i, \quad j \in \mathcal{J}_i^{[\ell]}, \quad (8)$$

where the $\sqrt{d_h}$ scaling is omitted, since the ranking is invariant under positive rescaling. The descent starts at the coarsest scale with every entry as a candidate, $\mathcal{J}_i^{[H-1]} = \{j : 0 \leq j < n_{H-1}\}$.

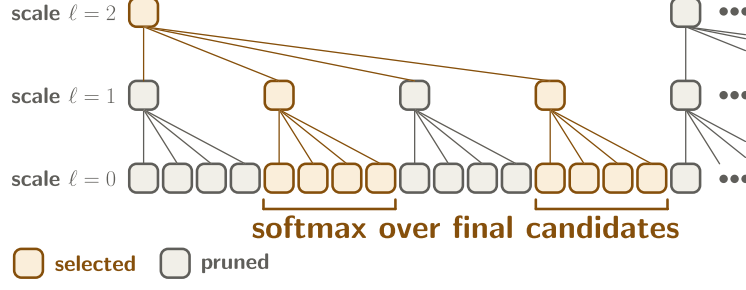


Figure 1: Recursive block selection over the hierarchical KV cache with $B = 4$.

Let $\mathcal{T}_i^{[\ell]} \subseteq \mathcal{J}_i^{[\ell]}$ denote the indices of the C/B largest scores in (8), with C a multiple of B . The children of these survivors form the candidate set at the next scale:

$$\mathcal{J}_i^{[\ell-1]} = \{jB + b : j \in \mathcal{T}_i^{[\ell]}, 0 \leq b < B\}, \quad (9)$$

where entries with $jB + b \geq n_{\ell-1}$ are masked, i.e. their scores are set to $-\infty$. Scoring and filtering repeat down to scale $\ell = 0$.

At $\ell = 0$, softmax attention is computed over the final candidate set, without further filtering:

$$a_{i,j} = \text{softmax}_{j \in \mathcal{J}_i^{[0]}} \left(\frac{k_j^\top q_i}{\sqrt{d_h}} \right), \quad (10)$$

and the per-head output is

$$y_i = \sum_{j \in \mathcal{J}_i^{[0]}} a_{i,j} v_j. \quad (11)$$

The proposed attention mechanism is illustrated in Figure 1.

3 Implementation

3.1 KV Cache

Unlike dense attention, the proposed RBA must store additional data: the composite keys. Scale $\ell = 0$ holds the keys and values produced by the projections in (1); scales $\ell = 1, \dots, H - 1$ hold the composite keys from (6); no composite values are formed.

The cache must support two operations:

1. appending keys and values, which entails updating the composite keys at all coarser scales;
2. retrieving keys and values for a given list of indices.

Retrieval in particular gathers from non-contiguous memory, which can add latency; the attention implementation must hide it.

Fixed context length. For a fixed context length, the cache can be realized as a ring buffer: new entries overwrite the oldest in place, bounding memory at the context size. When the buffer wraps, block boundaries no longer align with token order, so a composite near the write head sums a mix of the newest and oldest keys. By (5), the expected score of any relevant component in such a block is preserved, so selection is unaffected in expectation and no boundary realignment is needed.

3.2 Attention

The descent through the scales is independent across batches, heads, and queries, so it maps naturally onto a fused GPU kernel: each scale retrieves its composite keys, computes the scores under the causal mask, selects the C/B survivors, and expands their children into the next candidate set, with the softmax and head output computed directly at $\ell = 0$. The working set per descent is small –

the C candidate keys occupy $2d_h C$ bytes in BF16 – and fits comfortably in shared memory/SRAM, while the latency of the scattered key retrievals is hidden by resident warps scheduled across descents. The current implementation is pure PyTorch; a Triton kernel realizing this design, and with it timing comparisons against dense attention, is left to future work.

4 Complexity per Batch, Query, and Head

4.1 Memory

The number of stored vectors – assuming N is a multiple of B^{H-1} – is

$$\underbrace{N}_{\text{values}} + \underbrace{\sum_{\ell=0}^{H-1} \frac{N}{B^\ell}}_{\text{keys}} = N \left(1 + \frac{1 - B^{-H}}{1 - B^{-1}} \right) \leq 2N + \frac{N}{B-1}. \quad (12)$$

With d_h elements per vector at b_{type} bytes each, the total cache memory is

$$M \leq d_h b_{\text{type}} \left(2N + \frac{N}{B-1} \right). \quad (13)$$

The bound splits into the $2d_h b_{\text{type}} N$ of a standard key–value cache and the composite-key overhead. For $B = 8$,

$$M < 2.15 d_h b_{\text{type}} N, \quad (14)$$

an overhead below 7.5% – the hierarchy adds negligible memory atop a standard KV cache.

4.2 Memory Bandwidth

The descent issues C/B contiguous reads per scale – each of size $B \cdot d_h \cdot b_{\text{type}}$ bytes – plus one more such set for the values at $\ell = 0$. The total transferred is

$$(H+1) \cdot (C/B) \cdot B \cdot d_h \cdot b_{\text{type}} = (H+1) \cdot C \cdot d_h \cdot b_{\text{type}}. \quad (15)$$

Dense attention transfers $2N \cdot d_h \cdot b_{\text{type}}$ bytes for the same instance, so the method moves less data whenever $H+1 < 2N/C$. With $H = \lceil 1 + \log_B(N/C) \rceil$,

$$H+1 = 2 + \log_B(N/C) + \epsilon, \quad 0 \leq \epsilon < 1. \quad (16)$$

Since $\ln t \leq t - 1$, it follows that $\log_B(N/C) \leq N/C - 1$ for $B \geq 3$, so

$$H+1 \leq 1 + N/C + \epsilon < 2 + N/C < 2N/C, \quad N > 2C. \quad (17)$$

Hence, the proposed RBA moves less data whenever $N > 2C$, and the gap widens rapidly: at $N = 10^6$ with the experimental setting $B = 8$, $C = 256$, the descent reads $(H+1)C = 1536$ vectors against 2×10^6 for dense attention – a reduction by three orders of magnitude. The reads are block-contiguous but scattered across the cache, so the achieved bandwidth may fall short of the peak.

4.3 Compute

The compute-heavy operations per descent are:

1. attention scores: $\mathcal{O}(C \cdot d_h)$ per scale;
2. survivor selection: $\mathcal{O}(C \log C)$, or $\mathcal{O}(C \log^2 C)$ for a bitonic sort in Triton;
3. softmax: $\mathcal{O}(C)$;
4. output: $\mathcal{O}(C \cdot d_h)$.

In practice the bottleneck is the attention score (computed at every scale). The overall cost is $\mathcal{O}(H \cdot C \cdot d_h)$, and with $H = \lceil 1 + \log_B(N/C) \rceil$ this is

$$\mathcal{O}(C \cdot d_h \cdot \log N). \quad (18)$$

Dense attention costs $\mathcal{O}(N \cdot d_h)$ for the same instance. Treating C and d_h as constants, the gap is stark: $\mathcal{O}(\log N)$ for RBA versus $\mathcal{O}(N)$ for dense attention.

5 Experiment

The model Gemma-4-31B-it by Google DeepMind was used with BF16 precision [19, 20]. The model employs GQA with 4 key-value groups and 32 query heads (8 query heads share one key-value head), a head dimension of $d_h = 512$ – unusually large and above the maximum supported by FlashAttention – and interleaved local-global attention at a ratio of 5:1. The dense attention in the global layers (5, 11, 17, 23, 29, 35, 41, 47, 53, 59) was replaced by the proposed RBA; no training or fine-tuning was performed. The hyperparameters $C = 256$ and $B = 8$ were chosen manually based on preliminary tests on small sample sizes. The cache is implemented as a ring buffer (Section 3.1) with a capacity of 131 072 tokens – far above the maximum context length of 32 000 in the experiments – yielding a hierarchy depth of $H = 4$.

The model with RBA was compared to the vanilla model on the following needle-in-a-haystack (NIAH) tasks, run with opencompass [21–24]:

1. context lengths: 1000, 4000, 16000, 32000;
2. needle depths: 0%, 50%, 100%;
3. language: English;
4. tasks:
 - (a) Multi-Needle Reasoning (Ancestral Trace Challenge) with 2 and 4 hops (10 cases per configuration): Tables 1 and 2,
 - (b) Single-Needle Retrieval (10 cases per configuration): Table 3,
 - (c) Multi-Needle Retrieval with 3 needles placed at the depths above within each case (25 cases per context length): Table 4, where success is reported per needle, so the depth columns refer to needles within the same cases,

This gives 460 test cases per model. The original benchmark covers more needle counts, context lengths, and depths, and runs in two languages (en, zh); as the present evaluation is a proof of concept, only a fraction of the benchmark is used.

The reported benchmark ran on a single NVIDIA H200 SXM GPU (141 GB) in BF16 precision and consumed roughly 7 GPU-hours of run time (RBA 4.2 h, dense attention 2.7 h). The full project, including preliminary tests, the manual selection of C and B , and failed runs, consumed roughly 26 GPU-hours on cloud instances spanning several GPU types (H200, H100, L40S, and consumer cards). The wall-clock times are not performance measurements: the dense baseline used the shipped implementation, except that the context was fed in chunks of 256 tokens rather than as a single 32k sequence, whereas RBA used an unoptimized pure-PyTorch implementation that, moreover, always performs the work of the full 131 072-token cache regardless of the actual context length.

Table 1: Multi-Needle Reasoning (2 hops): accuracy in % over 10 cases per configuration, by context length N and needle depth in % of N . RBA matches or exceeds dense attention in every configuration, and its advantage widens as N grows, where dense attention degrades sharply.

Method	$N = 1000$			$N = 4000$			$N = 16000$			$N = 32000$		
	0	50	100	0	50	100	0	50	100	0	50	100
RBA	100	100	100	90	100	100	80	50	100	70	50	100
Dense Attention	80	80	80	40	50	70	10	0	50	0	0	40

Table 2: Multi-Needle Reasoning (4 hops): accuracy in % over 10 cases per configuration, by context length N and needle depth in % of N . RBA matches or exceeds dense attention in every configuration, but both degrade sharply with N , marking this as the hardest task.

Method	$N = 1000$			$N = 4000$			$N = 16000$			$N = 32000$		
	0	50	100	0	50	100	0	50	100	0	50	100
RBA	80	80	80	30	20	70	10	10	40	0	0	70
Dense Attention	30	30	30	10	20	40	0	0	20	0	0	20

Table 3: Single-Needle Retrieval: accuracy in % over 10 cases per configuration, by context length N and needle depth in % of N . RBA retrieves near-perfectly throughout, while dense attention collapses at mid-depth (50%) for $N \geq 16000$.

Method	$N = 1000$			$N = 4000$			$N = 16000$			$N = 32000$		
	0	50	100	0	50	100	0	50	100	0	50	100
RBA	100	100	100	100	100	100	100	100	100	100	90	100
Dense Attention	100	100	100	100	100	100	90	0	90	90	0	80

Table 4: Multi-Needle Retrieval: per-needle success in % over 25 cases per context length N ; each case contains all 3 needles, and columns give the depth of the scored needle. Dense attention again fails on mid-depth needles for $N \geq 16000$, where RBA remains accurate; both decline mildly at $N = 32000$.

Method	$N = 1000$			$N = 4000$			$N = 16000$			$N = 32000$		
	0	50	100	0	50	100	0	50	100	0	50	100
RBA	100	100	100	100	100	100	100	80	100	100	68	88
Dense Attention	100	100	100	100	88	96	96	12	100	96	12	88

6 Results, Discussion, and Limitations

Tables 1–4 show the results of the benchmark. The proposed RBA was applied as a drop-in replacement without any training and matched or exceeded the accuracy of dense attention in all 48 reported results; on the reasoning tasks the gains are substantial. The clearest difference appears for needles placed at depth 50%: at $N \geq 16000$, dense attention drops to 0–12% accuracy (Tables 3 and 4) – the known lost-in-the-middle effect – while RBA remains largely accurate. This observation is consistent with the attention dispersion discussed in the introduction: since the softmax is restricted to at most C keys, the attention weights remain concentrated, whereas dense attention spreads them over the entire context. The experiment, however, cannot isolate this mechanism. How RBA compares to other sparse attention methods also remains open.

With 10 cases per configuration (25 for Multi-Needle Retrieval), the individual results are coarse: a single case changes a value by 10 points (4 points for Multi-Needle Retrieval), so differences of one or two cases may well be noise. The improvement of RBA over dense attention, however, is real: RBA wins 33 of the 48 results and ties the remaining 15, losing none, frequently by margins of 30 points and more. Even though some results are correlated – the depth columns of Table 4 stem from the same cases – such a one-sided pattern cannot be explained by chance.

The computation time is not reported since the implementation of RBA is not optimized: the attention mechanism is implemented purely in PyTorch rather than as a Triton kernel. As a result, large intermediate tensors are written to and read back from HBM unnecessarily, and non-contiguous memory is accessed on a large scale. The baseline does not run at its optimum either: FlashAttention supports head dimensions only up to 256, so the model with $d_h = 512$ falls back to slower attention implementations.

Apart from expected failure modes – such as not finding the needle and outputting gibberish – which both variants share, the model with RBA occasionally appears to lose the relevant key–value pairs during generation and begins to doubt its so-far correct chain of thought. This seems to be a negative interaction between the sparse attention and the learned behavior of avoiding hallucinations: when the selection misses the needle at some decoding steps, the evidence disappears from the model’s view, and the model treats this as a sign that it hallucinated the information. If that is the case, sparse attention mechanisms might need end-to-end training after all. The available experiment does not allow a deeper analysis of this behavior.

In particular, the Multi-Needle Reasoning Task with 4 hops is difficult, as the model must correctly find all 4 needles and additionally reason over them to arrive at the answer. Accuracy drops starkly with context length, indicating that the attention mechanism degrades as context grows. This matches the caveat of Section 2.2: summarizing too large blocks leads to a poor representation of their content and therefore to a poor selection of relevant keys, which in turn leads to missing the needle. At shorter

contexts the hierarchical structure is the same, but the retained candidates cover a much larger fraction of the context, so the correct coarse-level blocks are likely selected even by chance; as the context grows, this fraction shrinks and the selection must rely on the composite keys. The experiment does not allow a deeper analysis of this mechanism.

Moreover, since summarizing larger blocks of keys appears difficult, it remains an open question whether hierarchies with $H > 2$ levels are viable: whether blocks of keys can be summarized into composite keys, and blocks of composites into even coarser composites, while it is still possible to correctly trace the relevant keys down the hierarchy. Fundamentally, the summarization methods (e.g. mean, sum, center, min-max, learned MLP) must be investigated and compared.

The interleaved attention of the model affects the interpretation of the results: RBA replaces only the 10 global layers, while the 50 local layers with their sliding window of 1024 tokens remain unchanged. A needle placed at depth 100% lies close to the end of the context and therefore within the local window of the positions that generate the answer, so it can be retrieved through local attention alone – independently of the global attention mechanism. This explains why the accuracy at depth 100% remains high for both variants; it also explains the otherwise surprising 70% of RBA at $N = 32000$ compared to 40% at $N = 16000$ in Table 2. Consequently, only the results at depths 0% and 50% actually test the replaced global attention.

In summary, the proposed approach outperforms dense attention in accuracy on the chosen NIAH benchmark in every configuration. It remains untested whether this advantage carries over to everyday use. It is also unclear whether the method actually reduces inference time; answering this question requires the Triton kernel discussed in Section 3.2. A comparison with other sparse attention methods is still missing, as is an evaluation at the context lengths of 10^5 – 10^6 tokens that motivate the method. The reason for the missing long-context evaluation lies with the baseline: without FlashAttention support for $d_h = 512$, dense attention runs out of memory well before such lengths, and implementing a custom FlashAttention variant is error-prone and was not an option as a baseline; RBA itself is not affected by this limit. Finally, the results suggest that the sum over blocks degrades as a selection signal at coarse scales, so it remains open whether hierarchies with more than two levels are viable in practice – even though exactly these deeper hierarchies enable the sublinear per-token cost.

Critically, the training-free variant presented here applies only to models pretrained with dense attention: a model trained with NSA, DSA, or MSA from the start offers no dense attention to replace. Should the field move to such natively trained sparse attention, the use case of the drop-in replacement narrows to the existing dense models.

The approach itself, however, is not tied to the training-free setting. Recursively summarizing keys and selecting relevant key-value pairs through the resulting hierarchy reduces the compute and memory access per token to $\mathcal{O}(\log N)$, compared to $\mathcal{O}(N)$ for dense attention. Flat sparse attention methods such as NSA, DSA, and MSA also remain linear per token, but their speed-up already enables production workloads at 10^6 tokens; the asymptotic advantage of the hierarchy can therefore only pay off at even longer contexts (e.g. persistent memory over sessions). For example, at $8\times$ the context length, flat sparse attention methods demand $8\times$ the compute and memory bandwidth, whereas the proposed method needs only one additional scale – a negligible cost in comparison. The additional memory requirement stays below 7.5% compared to dense attention, and in the experiment the method outperforms dense attention in every configuration without any training. The summarization method is the make-or-break concern: if summarizing at coarser scales proves impractical, this research direction collapses. A learned summarization trained end-to-end (Section 7) is the most promising remedy – such a summarizer would both sharpen the selection of relevant keys and carry the hierarchical selection over to natively trained models, yielding a sparse attention mechanism that stays fast and reliable regardless of context length.

7 Future Directions

Learned summarization. Similar to NSA [15] or DMC [4], groups of keys could be summarized by a learned MLP for use in the proposed RBA. Such a summary could retain the relevant information and discard the unimportant. At coarser levels, however, it eventually becomes impossible to retain all relevant information in a fixed-size vector. A further issue is that the keys relevant at inference are unknown at training time, which may bias the selection.

Exploration-based adaptation of the summarization. Extending the previous idea, the summarization could be adapted online. Instead of the top C/B positions at each scale, the top $C/B + E$ are selected, with a small exploration budget E . If a child of one of the E exploration positions reaches the top C/B at the next scale, its parent composite scored lower than its content warranted, and the summarization is adapted so that the composite better reflects that ranking. Since this adaptation is cheap and runs continuously, it could take place after deployment: a given user’s inputs tend to cover a limited range of content, so the summarization could specialize on exactly that range, improving the selection where it matters most.

Delayed update of the selection. Currently, every query attends to the composite keys directly. Since nearby keys are similar [16], nearby queries are plausibly similar too, so queries could be summarized in the same way. The selection would then be updated only once a composite query block is complete, and the set of selected keys would change less abruptly. This would also speed up inference, as the descent through the hierarchy runs once per query block rather than once per query. Moreover, a composite query carries more information than a single one, which could yield a better selection criterion at coarser levels. The risk is that a query which would trigger near one-hot attention enters the selection only with delay, missing the needle.

Combination with KV-eviction/compression strategies. The proposed RBA offers a valuable signal for deciding which tokens to keep and which to evict, so it combines naturally with KV-eviction methods. Moreover, when a group of tokens is evicted, its composite key could be retained as a proxy going forward; this would require summarizing the values before eviction, since RBA otherwise forms no composite values.

References

- [1] Petar Veličković, Christos Perivolaropoulos, Federico Barbero, and Razvan Pascanu. Softmax is not enough (for sharp size generalisation). *arXiv preprint arXiv:2410.01104*, 2024.
- [2] Ken M Nakanishi. Scalable-softmax is superior for attention. *arXiv preprint arXiv:2501.19399*, 2025.
- [3] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. In *International Conference on Learning Representations*, volume 2024, pages 31932–31951, 2024.
- [4] Piotr Nawrot, Adrian Łańcucki, Marcin Chochowski, David Tarjan, and Edoardo M Ponti. Dynamic memory compression: Retrofitting llms for accelerated inference. *arXiv preprint arXiv:2403.09636*, 2024.
- [5] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901. Association for Computational Linguistics, 2023.
- [6] William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelley. Reducing transformer key-value cache size with cross-layer attention. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 86927–86957. Curran Associates, Inc., 2024. doi: 10.52202/079017-2758. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/9e23d020c18e4c40d81c6a0fc7a46f68-Paper-Conference.pdf.
- [7] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning*, pages 31094–31116. PMLR, 2023.
- [8] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

- [9] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [10] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710, 2023.
- [11] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37:22947–22970, 2024.
- [12] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. *arXiv preprint arXiv:2406.10774*, 2024.
- [13] Weilin Zhao, Zihan Zhou, Zhou Su, Chaojun Xiao, Yuxuan Li, Yanghao Li, Yudi Zhang, Weilun Zhao, Zhen Li, Yuxiang Huang, et al. Infilmm-v2: Dense-sparse switchable attention for seamless short-to-long adaptation. *arXiv preprint arXiv:2509.24663*, 2025.
- [14] Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, et al. Moba: Mixture of block attention for long-context llms. *Advances in Neural Information Processing Systems*, 38:17790–17815, 2026.
- [15] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23078–23097, 2025.
- [16] Heejun Lee, Geon Park, Youngwan Lee, Jaduk Suh, Jina Kim, Wonyong Jeong, Bumsik Kim, Hyemin Lee, Myeongjae Jeon, and Sung Ju Hwang. A training-free sub-quadratic cost transformer model serving framework with hierarchically pruned attention. In *International Conference on Learning Representations*, volume 2025, pages 1280–1325, 2025.
- [17] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- [18] Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, et al. Minimax sparse attention. *arXiv preprint arXiv:2606.13392*, 2026.
- [19] Google DeepMind. Gemma 4. Model card, <https://deepmind.google/models/gemma/gemma-4/>, 2026. Apache 2.0 license. Accessed 2026-06-05.
- [20] Google DeepMind. Gemma 4 31B Instruction-Tuned (google/gemma-4-31B-it). Model weights via Hugging Face, <https://huggingface.co/google/gemma-4-31B-it>, 2026. Accessed 2026-06-05.
- [21] Mo Li, Songyang Zhang, Taolin Zhang, Haodong Duan, Yunxin Liu, and Kai Chen. Needlebench: Can llms do retrieval and reasoning in information-dense context?, 2025. URL <https://arxiv.org/abs/2407.11963>.
- [22] OpenCompass Contributors. Opencompass: A universal evaluation platform for foundation models. <https://github.com/open-compass/opencompass>, 2023.
- [23] gkamradt. Llmtest needle in a haystack - pressure testing llms. https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023.
- [24] Tianwen Wei, Liang Zhao, Lichang Zhang, Bo Zhu, Lijie Wang, Haihua Yang, Biye Li, Cheng Cheng, Weiwei Lü, Rui Hu, Chenxia Li, Liu Yang, Xilin Luo, Xuejie Wu, Lunan Liu, Wenjun Cheng, Peng Cheng, Jianhao Zhang, Xiaoyu Zhang, Lei Lin, Xiaokun Wang, Yutuan Ma, Chuanhai Dong, Yanqi Sun, Yifu Chen, Yongyi Peng, Xiaojuan Liang, Shuicheng Yan, Han Fang, and Yahui Zhou. Skywork: A more open bilingual foundation model, 2023.

A Benchmark Wall-Clock Times

Tables 5 and 6 report the wall-clock execution times of the benchmark.

Table 5: RBA wall-clock time (mm:ss) on a single NVIDIA H200 SXM, by task and context length N , summed over the three needle depths (0%, 50%, 100%); multi-needle retrieval has no depth split, as the three needles lie within each case. These are not performance measurements: the implementation is unoptimized pure PyTorch and always performs the work of the full 131 072-token cache regardless of N .

Task	$N=1000$	$N=4000$	$N=16000$	$N=32000$	Total
Multi-Needle Reasoning (2 hops)	4:51	6:22	19:42	43:43	74:38
Multi-Needle Reasoning (4 hops)	6:56	7:47	24:28	59:13	98:24
Single-Needle Retrieval	0:32	1:18	10:58	28:07	40:55
Multi-Needle Retrieval	1:11	1:50	10:28	25:55	39:24
Total	13:30	17:17	65:36	156:58	253:21

Table 6: Dense-attention wall-clock time (mm:ss) on a single NVIDIA H200 SXM, by task and context length N , summed over the three needle depths (0%, 50%, 100%); multi-needle retrieval has no depth split, as the three needles lie within each case. The dense baseline uses the shipped implementation with the context fed in chunks of 256 tokens, and lacks FlashAttention support at $d_h = 512$, so it is not representative of the state of the art. These are not performance measurements.

Task	$N=1000$	$N=4000$	$N=16000$	$N=32000$	Total
Multi-Needle Reasoning (2 hops)	8:22	3:46	9:55	24:43	46:46
Multi-Needle Reasoning (4 hops)	22:23	5:08	10:48	27:27	65:46
Single-Needle Retrieval	0:25	0:47	4:47	15:51	21:50
Multi-Needle Retrieval	2:37	1:08	4:59	19:57	28:41
Total	33:47	10:49	30:29	87:58	163:03